Interrupts

What is Interrupts?

- Interrupts allow you to **pause graph execution** at specific points and **wait for external input** before continuing.
- This enables human-in-the-loop patterns where you need external input to proceed with critical decisions or actions

Why Interrupts Needed?

With Interrupts:

- Controlled, pausable execution**.**
- Waits for **external input/completion of any dependent processes** before continuing to next step**.**
- Resumes from **exact pause point** after receiving input**.**

Without Interrupts

- Fully automated, **continuous execution.**
- No pause points for **human intervention.**

Example Use cases

- Email/communication before sending need **human approval.**
- Booking Flight Ticket via agent **need human intervention**
- Wait for payment confirmation webhook (Interrupts without HITL)

Pausing/Resuming Using Interrupts

What is Pausing?

- Stopping graph execution at a **specific point**.
- Graph waits indefinitely for external input
- **Execution does not continue** until resumed
- It uses **interrupt()** to pause graph execution.

Use Interrupt, We need

- **Checkpointer**: Must be configured (saves state) .
- **Thread ID**: Must provide in config (identifies the session)
- **JSON-serializable data**: Payload must be serializable

```python
from langgraph.types import interrupt

def approval_node(state: State):
    # Pause and ask for approval
    approved = interrupt("Do you approve this action?")

    # When you resume, Command(resume=...) returns that value here
    return {"approved": approved}
```

Code Explanation:

- **Graph execution gets suspended** at the exact point where **interrupt()** is called.
- **State is saved** using the checkpointer.
- **Value is returned** to the caller under __interrupt__; it can be any JSON-serializable value (string, object, array, etc.)
- **Graph waits indefinitely** until you resume execution with a response
- **Response is passed back** into the node when you resume, becoming the return value of the interrupt() call

Resuming Using Interrupts:

- After an interrupt pauses execution, you resume the graph by invoking it again with a **Command()** that contains the resume value.
- Allowing the node to continue execution with the external input.
- The resume value is passed back to the interrupt.

```
graph.invoke(Command(resume=True), config=config)
```

Sample Program:

```python
from typing import TypedDict
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class BookingState(TypedDict):
    flight: str
    price: int
    confirmed: bool

def booking_node(state: BookingState):
    # Pause and show booking details
    approved = interrupt({
        "question": "Confirm booking?",
        "flight": state["flight"],
        "price": state["price"]
    })

    return {"confirmed": approved}

# Build graph
builder = StateGraph(BookingState)
builder.add_node("booking", booking_node)
builder.add_edge(START, "booking")
builder.add_edge("booking", END)

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# Step 1: Run graph
config = {"configurable": {"thread_id": "booking-123"}}
result = graph.invoke({
    "flight": "AI-205",
    "price": 450,
    "confirmed": False
}, config=config)

# Step 2: Check interrupt and ask user
print(result["__interrupt__"])
# [Interrupt(value={'question': 'Confirm booking?', ...})]

def ask_user_confirmation():
    response = input("Confirm booking? (yes/no): ").strip().lower()
    return response == "yes"

# Step 3: Get user input and resume
user_approved = ask_user_confirmation()

if user_approved:
    final = graph.invoke(Command(resume=user_approved), config=config)
    print(f"Flight {result['flight']} is booked!")
else:
    print("Flight booking cancelled.")
```

Code Explanation:

Step 1: Build the Graph

```
builder = StateGraph(BookingState)
builder.add_node("booking", booking_node)
builder.add_edge(START, "booking")
builder.add_edge("booking", END)
```

- Build the Graph with Start and end node with checkpointers

Step 2: Run the Graph

```
config = {"configurable": {"thread_id": "booking-123"}}
result = graph.invoke({
    "flight": "AI-205",
    "price": 450,
    "confirmed": False
}, config=config)
```

- Run the Graph with thread_id

Step 3: Checking for the Interrupts

```
def booking_node(state: BookingState):
    # Pause and show booking details
    approved = interrupt({
        "question": "Confirm booking?",
        "flight": state["flight"],
        "price": state["price"]
    })

    return {"confirmed": approved}
```

Code Explanation:

- Waiting for the approval to execute further

Step 5: Resuming the Interrupt

```
def ask_user_confirmation():
    response = input("Confirm booking? (yes/no): ").strip().lower()
    return response == "yes"

# Step 3: Get user input and resume
user_approved = ask_user_confirmation()

if user_approved:
    final = graph.invoke(Command(resume=user_approved), config=config)
    print(f"Flight {result['flight']} is booked!")
```

- Resuming the interrupt using Command method()

Output:

```
[Interrupt(value={'question': 'Confirm booking?', 'flight': 'AI-205', 'price': 450}, id='e093ed2adcc4947d7cbbd3b007ebeb1c')]
Confirm booking? (yes/no): yes
Flight AI-205 is booked!
```

Common Patterns

Common Patterns in Interrupts:

- Stream with human-in-the-loop (HITL) interrupts.
- Approve or Reject
- Review and edit State
- Interrupts in tools
- Validating human input

Stream with human-in-the-loop(HITL):

- Detect interrupts in real-time during streaming execution for **conversational.**

Why we need ?

- Users expect real-time typing indicators during LLM generation.
- Interrupts should trigger **during generation**.

Example Use case:

- Customer support chatbot: This cancellation triggers $500 fee approve?.

Approve or Reject:

- Pause before making permanent changes and wait for a person to give clear approval..

Why we need?

- Prevents autonomous execution of high-risk operations (deletions, payments, API calls)
- Provides audit trail of human decisions

Example Use case:

- Financial services platform

Review and Edit:

- humans refine AI-generated content before it impacts downstream systems**.**

Why we need ?

- Avoids "reject-and-regenerate" cycles by enabling direct edits.

Example Use case:

- Healthcare provider: AI drafts patient dis-charge instructions-> Nurses edit for clinical accuracy?.

Interrupt Inside Tools:

- Add safety rails at the exact moment of tool execution—not just before tool selection.

Why we need?

- Agents might select appropriate tools but with incorrect parameters
- Enables parameter correction during execution

Example Use case:

- Marketing platform- Agents sends mail to customers, interrupt and check the customer mail address

Input Validation Loop:

- Repeatedly prompt until receiving valid input that meets business rules.

Why we need ?

- Some workflows require strict data validation (KYC, financial forms).
- Avoids workflow failure due to bad inputs.

Example Use case:

- Cance Cab Booking Agents: User enters cab_id wrongly like ‘A1121’ Instead of ‘A1211’ .

Rules for Interrupts:

- **Do not wrap** interrupt **calls in try/except**.
  - When we wrap interrupt method() with try/except after pause it will throw special exception.
  - The interrupt won't propagate to the runtime.
  - Bad Practice:

```python
def node_a(state: State):
    try:
        interrupt("What's your name?")  # ❌ Will catch the interrupt exception!
    except Exception as e:
        print(e)
    return state
```

  - Good Practice:

```python
def node_a(state: State):
    # ✅ Good: interrupting first, then handling
    # error conditions separately
    interrupt("What's your name?")
    try:
        fetch_data()  # This can fail
    except Exception as e:
        print(e)
    return state
```

Rules for Interrupts:

- **Do not reorder** interrupt **calls within a node**.
  - LangGraph matches resume values to interrupts using **strictly index-based matching**. The order of interrupt calls must remain consistent across all executions of the node.
  - Good Practice:

```python
def node_a(state: State):
    name = interrupt("What's your name?")
    age = interrupt("What's your age?")
    city = interrupt("What's your city?")
    return {"name": name, "age": age, "city": city}
```

  - Bad Practice:

```python
def node_a(state: State):
    name = interrupt("What's your name?")

    # ❌ Conditionally skipping interrupts causes index mismatch on resume
    if state.get("needs_age"):
        age = interrupt("What's your age?")

    city = interrupt("What's your city?")
    return {"name": name, "city": city}
```

- **Do not return complex values in** interrupt **calls**
  - Interrupt values must be serializable since they're stored in the checkpoint.
  - Pass dictionaries/objects with simple values

Rules for Interrupts:

- Good Practice:

```python
def node_a(state: State):
    # ✅ Good: passing simple types that are serializable
    name = interrupt("What's your name?")
    count = interrupt(42)
    approved = interrupt(True)

    return {"name": name, "count": count, "approved": approved}
```

- Bad Practice:

```python
def validate_input(value):
    return len(value) > 0

def node_a(state: State):
    # ❌ Bad: passing a function to interrupt
    # The function cannot be serialized
    response = interrupt({
        "question": "What's your name?",
        "validator": validate_input  # This will fail
    })
    return {"name": response}
```

- **Side effects called before** interrupt **must be idempotent**
  - Since the node restarts from the beginning on resume, any side effects (API calls, DB writes) before an interrupt will execute multiple times.
  - Use idempotent operations (e.g., upsert instead of create)